



Chapter 2

Moving Forward with Logistic Regression

IN THIS CHAPTER

- » Considering the uses for logistic regression
- » Using logistic regression to guess classes
- » Using logistic regression to calculate probabilities
- » Employing multiclass logistic regression

.....

Linear regression, as described in the previous chapter, has limits. It helps you find only certain kinds of equations based on your data. To move forward in your ability to analyze data, you need logistic regression because it can help you model data with greater complexity. Of course, the modeling process itself is more complex as a result. The first part of this chapter helps you understand the origins of logistic regression and consider how it differs from linear regression so that you know which kind of regression to apply to particular datasets.

The second part of this chapter considers two uses for logistic regression: guessing the class of a particular object and the probability of something like an event. You find that these two uses are actually two sides of the same coin, in a way. In some respects, it comes down to

how you view the analysis you perform and how you employ it against new data.

Even though a single chapter can't possibly exhaust the topic of logistic regression, the third part of the chapter takes you a little deeper by exploring multiclass logistic regression. Instead of simply telling you that an object is of one class or another, multiclass logistic regression can help you determine which of multiple classes reflects the nature of an object.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the

`DSPD_0302_Logistic_Regression.ipynb` source code file for Python and the `DSPD_R_0302_Logistic_Regression.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Considering the History of Logistic Regression

Some kinds of data don't neatly fit in a straight line. In the previous chapter of this minibook, you read about the history of linear regression, which is also the start of the history for logistic regression. In that history, you discover that data represents the answer to a question that comes in the form of an equation, and that linear regression helps you locate the question, the equation, itself. You're still looking for an equation with logistic regression, but now you're looking for a more flexible equation, one that can handle data that doesn't express itself in the form of a straight line.

The original use for logistic regression was determining population growth. In 1789, Thomas Robert Malthus published a book entitled *An Essay on the Principle of Population*, in which he postulated that uncontained human population growth would represent a geometric

progression. More than 200 years later, this principle is still used in many settings for things like economic analysis (see the article at <https://www.intelligenteconomist.com/malthusian-theory/> for details). However, the key word here is *uncontained*. Population growth is seldom left to itself, without outside influence. The result would be impossibly high numbers that could never model how population growth actually works over the long term.



REMEMBER Consequently, in a series of three papers from 1838 to 1847, Pierre François Verhulst, under guidance from his mentor Adolphe Quetelet, created a better model to describe population growth using the logistic function. Some contention exists as to where the name *logistic* comes from, but it apparently strives to create a contrast with a logarithmic function. When working with a logistic function, the curve follows these steps:

1. The curve begins with exponential growth because the object being modeled has both unlimited resources and unlimited area in which to grow.
2. As saturation of the environment and depletion of resources occurs, growth takes on a linear form.
3. At maturity, when there is little room to grow and fewer resources to use, growth tends toward stopping and possibly does stop.

The logistic function model didn't happen in a vacuum. Verhulst modeled it on French, Belgian, Essex, and Russian population growth. In other words, he started with the data and created an equation to act as a question for the data.

Oddly enough, the logistic function didn't take off immediately; instead, it languished until someone else discovered a need for it. In 1920, Raymond Pearl (who had just been appointed Director of Biometry and Vital Statistics at Johns Hopkins University) and Lowell J. Reed published a paper on the food needs of a growing population, entitled *On the Rate of Growth of the Population of the United States*

since 1790 and Its Mathematical Representation. This is actually just the first of a whole series of papers that would investigate issues like longevity, fertility, contraception, and the effects of smoking and alcohol. The history of the logistic function and logistic regression associated with it is long and complex, but you can find a relatively complete treatment of the topic at

<https://papers.tinbergen.nl/02119.pdf>.

Differentiating between Linear and Logistic Regression

Both linear and logistic regression see a lot of use in data science but are commonly used for different kinds of problems. You need to know and understand both types of regression to perform a full range of data science tasks. Of the two, logistic regression is harder to understand in many respects because it necessarily uses a more complex equation model. The following sections give you a basic overview of how linear and logistic regression differ so that you can better understand the information in the rest of the chapter.

Considering the model

Any discussion of the difference between linear and logistic regression must start with the underlying equation model. The equation for linear regression is straightforward, as discussed in Book 3, [Chapter 1](#):

$$y = a + bx$$

You may see this equation in other forms and you may see it called ordinary least-squares regression, but the essential concept is always the same. Depending on the source you use, some of the equations used to express logistic regression can become downright terrifying unless you're a math major. However, the start of this discussion can use one of the simplest views of logistic regression:

$$p = f(a + bx)$$

This equation model says that the probability of an occurrence, p , is equal to the logistic function, f , applied to two model parameters, a and b , and one explanatory variable, x . When you look at this particular model, you see that it really isn't all that different from the linear regression model, except that you now feed the result of the linear regression through the logistic function to obtain the required curve. The output (dependent variable) is a probability ranging from 0 (not going to happen) to 1 (definitely will happen), or a categorization that says something is either part of the category or not part of the category. (You can also perform multiclass categorization, as described later in this chapter, but focus on the binary response for now.) The best way to view the difference between linear regression output and logistic regression output is to say the following:

- **Linear regression is continuous.** A continuous value can take any value within a specified interval (range) of values. For example, no matter how closely the height of two individuals matches, you can always find someone whose height fits between those two individuals. Examples of continuous values include:
 - Height
 - Weight
 - Waist size
- **Logistic regression is discrete.** A discrete value has specific values that it can assume. For example, a hospital can admit only a specific number of patients in a given day. You can't admit half a patient (at least, not alive). Examples of discrete values include:
 - Number of people at the fair
 - Number of jellybeans in the jar
 - Colors of automobiles produced by a vendor

Defining the logistic function

Of course, now you need to know about the logistic function. You can find a variety of forms of this function as well, but here's the easiest

one to understand:

$$f(x) = e^x / e^x + 1$$

You already know about f , which is the logistic function, and x equals the algorithm you want to use, which is $a + bx$ in this case. That leaves e , which is the natural logarithm and has an irrational value of 2.718, for the sake of discussion (you can see a better approximation of the whole value at

<https://www.intmath.com/exponential-logarithmic-functions/5-logs-base-e-ln.php>). Another way you see this function expressed is

$$f(x) = 1 / (1 + e^{-x})$$



REMEMBER Both forms are correct, but the first form is easier to use.

Consider a simple problem in which a , the y-intercept, is 0, and b , the slope, is 1. The example uses x values from -6 to 6. Consequently, the first $f(x)$ value would look like this when calculated (all values are rounded):

$$(1) e^{-6} / (1 + e^{-6})$$

$$(2) 0.00248 / 1 + 0.00248$$

$$(3) 0.002474$$

As you might expect, an x value of 0 would result in an $f(x)$ value of 0.5, and an x value of 6 would result in an $f(x)$ value of 0.9975.

Obviously, a linear regression would show different results for precisely the same x values. If you calculate and plot all the results from both logistic and linear regression using the following code, you receive a plot like the one shown in [Figure 2-1](#).

```
import matplotlib.pyplot as plt

%matplotlib inline

from math import exp

x_values = range(-6, 7)

lin_values = [(0 + 1*x) / 13 for x in range(0, 13)]

log_values = [exp(0 + 1*x) / (1 + exp(0 + 1*x))

               for x in x_values]

plt.plot(x_values, lin_values, 'b-^')

plt.plot(x_values, log_values, 'g-*')

plt.legend(['Linear', 'Logistic'])

plt.show()
```

This example relies on list comprehension

(<https://www.pythonforbeginners.com/basics/list-comprehensions-in-python>) to calculate the values because it makes the calculations clearer. The linear regression uses a different numeric range because you must normalize the values to appear in the 0 to 1 range for comparison. This is also why you divide the calculated values by 13. The `exp(x)` call used for the logistic regression raises `e` to the power of `x`, e^x , as needed for the logistic function.

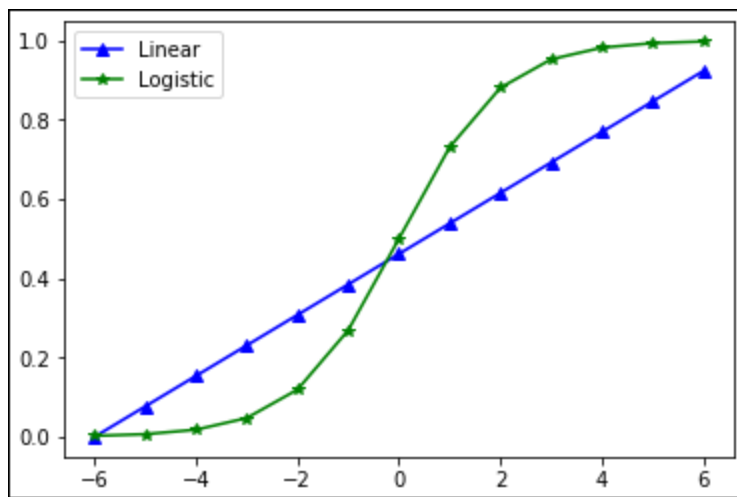


FIGURE 2-1: Contrasting linear to logistic regression.



WARNING The model discussed here is simplified, and some math majors out there are probably throwing a temper tantrum of the most profound proportions right now. The Python or R package you use will actually take care of the math in the background, so really, what you need to know is how the math works at a basic level so that you can understand how to use the packages. This section provides what you need to use the packages. However, if you insist on carrying out the calculations the old way, chalk to chalkboard, you'll likely need a lot more information.

Understanding the problems that logistic regression solves

You can separate logistic regression into several categories. The first is simple logistic regression, in which you have one dependent variable and one independent variable, much as you see in simple linear regression. However, because of how you calculate the logistic regression, you can expect only two kinds of output:

- **Classification:** Decides between two available outcomes, such as male or female, yes or no, or high or low. The outcome is dependent on which side of the line a particular data point falls.

- **Probability:** Determines the probability that something is true or false. The values true and false can have specific meanings. For example, you might want to know the probability that a particular apple will be yellow or red based on the presence of yellow and red apples in a bin.

Fitting the curve

As part of understanding the difference between linear and logistic regression, consider the grade prediction problem from the previous chapter, which lends itself well to linear regression. In the following code, you see the effect of trying to use logistic regression with that data:

```
x1 = range(0,9)
```

```
y1 = (0.25, 0.33, 0.41, 0.53, 0.59,
```

```
0.70, 0.78, 0.86, 0.98)
```

```
plt.scatter(x1, y1, c='r')
```

```
lin_values = [0.242 + 0.0933*x for x in x1]
```

```
log_values = [exp(0.242 + .9033*x) /
```

```
(1 + exp(0.242 + .9033*x))
```

```
for x in range(-4, 5)]
```

```
plt.plot(x1, lin_values, 'b-^')
```

```
plt.plot(x1, log_values, 'g-*')
```

```
plt.legend(['Linear', 'Logistic', 'Org Data'])
```

```
plt.show()
```

The example has undergone a few changes to make it easier to see precisely what is happening. It relies on the same data that was converted from questions answered correctly on the exam to a percentage. If you have 100 questions and you answer 25 of them correctly, you have answered 25 percent (0.25) of them correctly. Instead of performing the actual analysis, the code uses the equation found in the “[Working through simple linear regression](#)” section of Book 3, [Chapter 1](#). The values are normalized to produce values between 0 and 1 percent. [Figure 2-2](#) shows the output of this experiment.

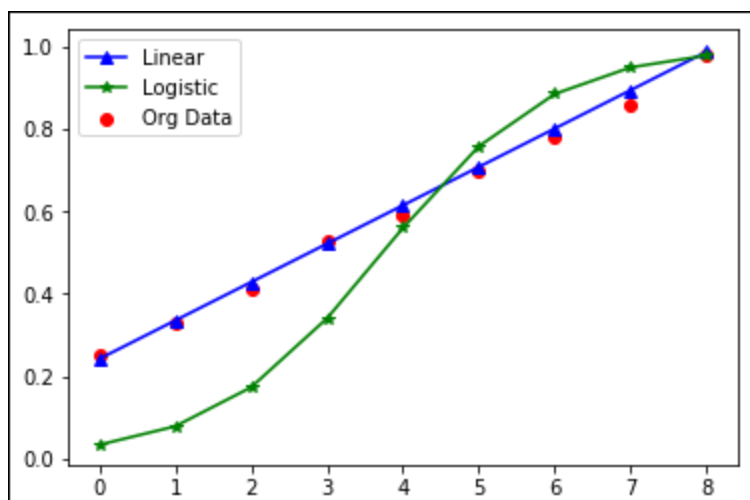


FIGURE 2-2: Considering the approach to fitting the data.

As you can see from the figure, the linear regression follows the data points closely. The logistic regression doesn't. However, logistic regression often is the correct choice when the data points naturally follow the logistic curve, which happens far more often than you might think. You must use the technique that fits your data best, which means using linear regression in this case.

Considering a pass/fail example

An essential point to remember is that logistic regression works best for probability and classification. Consider that points on an exam ultimately predict passing or failing the course. If you get a certain percentage of the answers correct, you pass, but you fail otherwise. The following code considers the same data used for the example in the previous section, but converts it to a pass/fail list. When a student gets at least 70 percent of the questions correct, success is assured.

```
y2 = [0 if x < 0.70 else 1 for x in y1]
```

```
plt.scatter(x1, y2, c='r')
```

```
lin_values = [0.242 + 0.0933*x for x in x1]
```

```
log_values = [exp(0.242 + .9033*x) /
```

```
(1 + exp(0.242 + .9033*x))
```

```
for x in range(-4, 5)]
```

```
plt.plot(x1, lin_values, 'b-^')
```

```
plt.plot(x1, log_values, 'g-*')
```

```
plt.legend(['Linear', 'Logistic', 'Org Data'])
```

```
plt.show()
```

This is an example of how you can use list comprehensions in Python to obtain a required dataset or data transformation. The list comprehension for `y2` starts with the continuous data in `y1` and turns it into discrete data. Note that the example uses precisely the same equations

as before. All that has changed is the manner in which you view the data, as shown in [Figure 2-3](#).

Because of the change in the data, linear regression is no longer the option to choose. Instead, you use logistic regression to fit the data. Take into account that this example really hasn't done any sort of analysis to optimize the results. The logistic regression fits the data even better if you do so (as you see in the sections that follow).

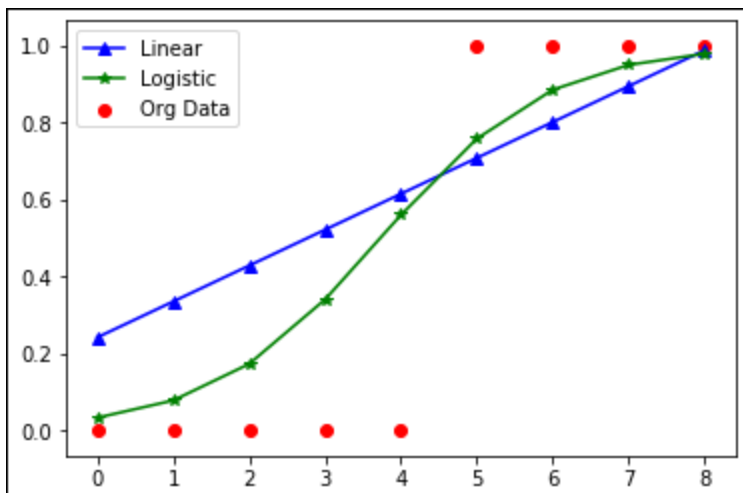


FIGURE 2-3: Contrasting linear to logistic regression.

Using Logistic Regression to Guess Classes

As noted earlier in the chapter, linear regression is well suited for estimating values, but it isn't the best tool for predicting the class of an observation. The previous sections focus on helping you understand the difference between linear and logistic regression. However, the examples use an exceptionally simple dataset to get the job done, which isn't what you encounter in the real world. The following sections use a larger dataset to help you better understand the nuances of working with logistic regression. Even though this dataset is still smaller than real-world counterparts, it provides enough data for you to begin understanding how to use logistic regression in a real-world setting.

Applying logistic regression

Logistic regression is similar to linear regression, with the only difference being the `y` data, which should contain integer values indicating the class relative to the observation. Using the Iris dataset from the Scikit-learn `datasets` module, you can use the values 0, 1, and 2 to denote three classes that correspond to three species:

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
X = iris.data[:-1,:]
```

```
y = iris.target[:-1]
```

To make the example easier to work with, leave a single value out so that later you can use this value to test the efficacy of the logistic regression model on it:

```
from sklearn.linear_model import LogisticRegression
```

```
logistic = LogisticRegression()
```

```
logistic.fit(X, y)
```

```
single_row_pred = logistic.predict(
```

```
    iris.data[-1, :].reshape(1, -1))
```

```
single_row_pred_proba = logistic.predict_proba(
```

```
    iris.data[-1, :].reshape(1, -1))
```

```
print ('Predicted class %s, real class %s'
```

```
      % (single_row_pred, iris.target[-1]))
```

```
print ('Probabilities for each class from 0 to 2: %s'
```

```
% single_row_pred_proba)
```

The preceding code snippet outputs the following:

```
Predicted class [2], real class 2
```

```
Probabilities for each class from 0 to 2:
```

```
[[ 0.00168787  0.28720074  0.71111138]]
```

In contrast to linear regression, logistic regression doesn't just output the resulting class (in this case, the class `2`) but also estimates the probability of the observation's being part of all three classes. Based on the observation used for prediction, logistic regression estimates a probability of 71 percent of its being from class `2` — a high probability, but not a perfect score, therefore leaving a margin of uncertainty.



TIP

Using probabilities lets you guess the most probable class, but you can also order the predictions with respect to being part of that class. This is especially useful for medical purposes: Ranking a prediction in terms of likelihood with respect to others can reveal what patients are at most risk of getting or already having a disease.

Considering when classes are more

The problem considered in the previous section concerning logistic regression automatically handles a multiple class problem (it started with three iris species to guess). Most algorithms provided by Scikit-learn that predict probabilities or a score for class can automatically handle multiclass problems using two different strategies:

- **One-Versus-Rest (OVR):** The algorithm compares every class with all the remaining classes, building a model for every class. If you have ten classes to guess, you have ten models. This approach relies on the

`OneVsRestClassifier` class from Scikit-learn. You might also see this strategy referred to as One-Versus-All (OVA).

- **One-Versus-One (OVO):** The algorithm compares every class against every individual remaining class, building a number of models equivalent to $n * (n-1) / 2$, where n is the number of classes. If you have ten classes, you have 45 models, $10 * (10 - 1) / 2$. This approach relies on the `OneVsOneClassifier` class from Scikit-learn.

In the case of logistic regression, the default multiclass strategy is the one versus the rest. The example in this section shows how to use both strategies with the handwritten digit dataset, containing a class for numbers from 0 to 9. The following code loads the data and places it into variables:

```
from sklearn.datasets import load_digits
```

```
digits = load_digits()
```

```
train = range(0, 1700)
```

```
test = range(1700, len(digits.data))
```

```
X = digits.data[train]
```

```
y = digits.target[train]
```

```
tX = digits.data[test]
```

```
ty = digits.target[test]
```

The observations are actually a grid of pixel values. The grid's dimensions are 8 pixels by 8 pixels. To make the data easier to learn by machine learning algorithms, the code aligns them into a list of 64 elements. The example reserves a part of the available examples for a test:

```
from sklearn.multiclass import OneVsRestClassifier
```

```
from sklearn.multiclass import OneVsOneClassifier

OVR = OneVsRestClassifier(LogisticRegression()).fit(X, y)

OVO = OneVsOneClassifier(LogisticRegression()).fit(X, y)

print('One vs rest accuracy: %.3f' % OVR.score(tX, ty))

print('One vs one accuracy: %.3f' % OVO.score(tX, ty))
```

The performances of the two multiclass strategies are

```
One vs rest accuracy: 0.938
```

```
One vs one accuracy: 0.969
```

The two multiclass classes `OneVsRestClassifier` and `OneVsOneClassifier` operate by incorporating the estimator (in this case, `LogisticRegression`). After incorporation, they usually work just like any other learning algorithm in Scikit-learn. Interestingly, the one-versus-one strategy obtained the highest accuracy thanks to its high number of models in competition.

Defining logistic regression performance

A logistic regression model will perform in a certain way. You split the data into two parts: training and testing. During training, the model will fine-tune the weights used to determine the classification or probability of like data. During testing, you verify the accuracy of the model. By using data from other datasets, you can further refine your estimation of the accuracy of the model in detecting the class of a particular object or of determining the probability of an object belonging to a certain class. Accuracy determines three things:

- The number of correct guesses, which includes both true positives (something is part of the class) and true negatives (something is not part of the

class)

- The number of false positives, with a guess saying that an object is part of a class, but it isn't
- The number of false negatives, with a guess saying that an object isn't part of a class, but it really is

To determine the *accuracy* of a model, you divide the number of correct guesses by the total number of objects. For example, an application might say that 80 objects are part of the class, but only 70 of these objects actually are. It might also say that 20 objects aren't part of the class, yet 30 of the objects actually are. The total number of objects is 100, so the accuracy is as follows: 70 true positives + 20 true negatives / 100 object total, or 90 percent.

However, you often see different measures used to define performance because accuracy is too inclusive. When you see the term *precision*, what it really means is the number of true positives divided by the number of total positives, and it doesn't account for true negatives or false negatives. For example, if an application says that 80 out of 100 objects are part of the class, yet it has guessed incorrectly in 10 of those cases, the precision is 70 true positives / 70 true positives + 10 false positives, or 87.5 percent. What this measure really asks is how many of the selected items are relevant to the analysis.

You also see the term *recall* used, which is the measure of the relevance of the selected items. In this case, you begin with the number of true positives, which is 70 in the example, and divide it by a combination of the true positives and the false negatives. In this case, the recall is 70 / 70 + 0, or 100 percent. An inverse relationship exists between recall and precision; to get better recall, you must normally trade precision. (The article at <https://towardsdatascience.com/precision-vs-recall-386cf9f89488> describes the relationships between accuracy, precision, and recall.) The examples at <https://www.kaggle.com/pablovargas/comparing-logistic-regression-performance> help you see how these three measures are used in real-world situations.



WARNING Remember that you don't ever get any form of analysis without cost. As you train with more examples and perform additional verification, the time required to create a model and maintain it for changing conditions increases. At some point, your model becomes incredibly accurate, but the cost in time is so great that the answer you receive is no longer relevant. So, another indicator of performance is timeliness; the task must proceed fast enough to ensure that the answer is still useful.

Switching to Probabilities

Up to now, the chapter has considered only regression models, which express numeric values as outputs from data learning. Most problems, however, also require classification. The following sections talk about how you can address both numeric and classification output.

Specifying a binary response

A solution to a problem involving a binary response (the model has to choose from between two possible classes) would be to code a response vector as a sequence of ones and zeros (or positive and negative values). The following Python code proves both the feasibility and limits of using a binary response:

```
import numpy as np
```

```
a = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

```
b = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(8,1)
```

```
from sklearn.linear_model import LinearRegression
```

```
regression = LinearRegression()
```

```
regression.fit(b,a)
```

```
print (regression.predict(b)>0.5)
```

When you run this code, you see the following output:

```
[False False False False  True  True  True  True]
```

In statistics, linear regression can't solve classification problems because doing so would create a series of violated statistical assumptions. So, for statistics, using regression models for classification purposes is mainly a theoretical problem, not a practical one. When performing tasks such as deep learning, the problem with linear regression is that it serves as a linear function that's trying to minimize prediction errors; therefore, depending on the slope of the computed line, it may not be able to solve the data problem.

When a linear regression is given the task of predicting two values, such as 0 and +1, which represent two classes, it tries to compute a line that provides results close to the target values. In some cases, even though the results are precise, the output is too far from the target values, which forces the regression line to adjust in order to minimize the summed errors. The change results in fewer summed deviance errors but more misclassified cases.

Linear regression doesn't produce acceptable results when the priority is classification accuracy, as shown in [Figure 2-4](#) on the left.

Therefore, it won't work satisfactorily in many classification tasks.

Linear regression works best on a continuum of numeric estimates.

However, for classification tasks, you need a more suitable measure, such as the probability of class ownership.

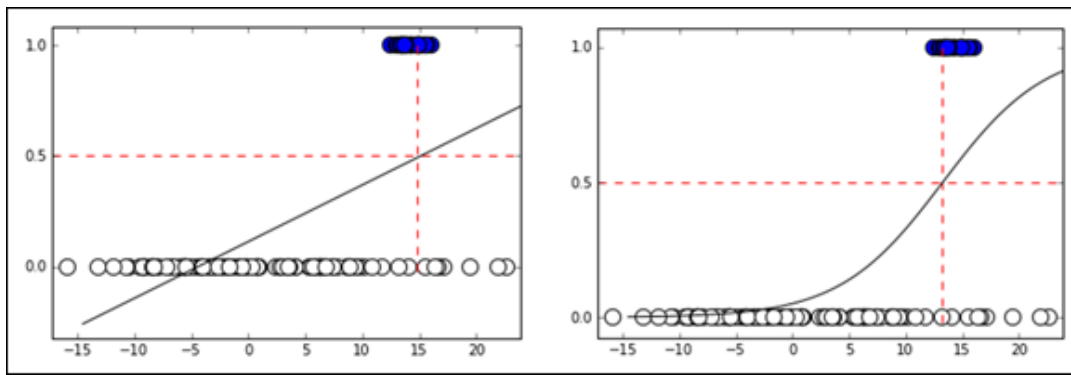


FIGURE 2-4: Probabilities do not work as well with a straight line as they do with a sigmoid curve.

Transforming numeric estimates into probabilities

The “[Defining the logistic function](#)” section, earlier in this chapter, describes how logistic regression differs from linear regression using the logistic function. In this function, the target is the probability that the response $f(x = 1)$ will correspond to the class 1. The letter x is the *regression result*, the sum of the variables weighted by their coefficients. The exponential function, $\exp(x)$, corresponds to Euler's number e elevated to the power of x . A linear regression using this transformation formula (also called a *link function*) for changing its results into probabilities is a logistic regression.

Logistic regression (shown on the right in [Figure 2-4](#)) is the same as a linear regression except that the y data contains integer numbers indicating the class that's relative to the observation. So, using the Boston dataset from the Scikit-learn `datasets` module, you can try to guess what makes houses in an area overly expensive (median values ≥ 40). The first step is to load the Boston dataset, `t` as shown here:

```
from sklearn.datasets import load_boston
```

```
from sklearn.preprocessing import scale
```

```
boston = load_boston()
```

```
X, y = scale(boston.data), boston.target
```

Now that you have the dataset loaded, you can perform the analysis:

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split
```

```
binary_y = np.array(y >= 40).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
```

```
    binary_y, test_size=0.33, random_state=5)
```

```
logistic = LogisticRegression()
```

```
logistic.fit(X_train,y_train)
```

```
from sklearn.metrics import accuracy_score
```

```
print('In-sample accuracy: %0.3f' %
```

```
    accuracy_score(y_train, logistic.predict(X_train)))
```

```
print('Out-of-sample accuracy: %0.3f' %
```

```
    accuracy_score(y_test, logistic.predict(X_test)))
```

Here's the output from this example:

```
In-sample accuracy: 0.973
```

```
Out-of-sample accuracy: 0.958
```

The example splits the data into training and test sets, enabling you to check the efficacy of the logistic regression model on data that the model hasn't used for learning. The resulting coefficients tell you the

probability of a particular class's being in the target class (which is any class encoded using a value of 1). If a coefficient increases the likelihood, it will have a positive coefficient; otherwise, the coefficient is negative.

```
for var,coef in zip(boston.feature_names,  
                    logistic.coef_[0]):  
    print ("%7s : %7.3f" %(var, coef))
```

The output shows the effect of each coefficient (not necessarily in order):

CRIM : -0.006

ZN : 0.197

INDUS : 0.580

CHAS : -0.023

NOX : -0.236

RM : 1.426

AGE : -0.048

DIS : -0.365

RAD : 0.645

TAX : -0.220

PTRATIO : -0.554

B : 0.049

```
LSTAT : -0.803
```

Reading the results on your screen, you can see that in Boston, criminality (CRIM) has some effect on prices. However, the level of poverty (LSTAT), distance from work (DIS), and pollution (NOX) all have much greater effects. Moreover, contrary to linear regression, logistic regression doesn't simply output the resulting class (in this case a 1 or a 0) but also estimates the probability of the observation's being part of one of the two classes:

```
print('\nClasses:', logistic.classes_)
```

```
print('\nProbs:\n', logistic.predict_proba(X_test)[:3,:])
```

The point is that you get a great deal more information based on the data you provide, as shown here:

```
Classes: [0 1]
```

```
Probs:
```

```
[[ 0.39022779  0.60977221]
```

```
[ 0.93856655  0.06143345]
```

```
[ 0.98425623  0.01574377]]
```

In this small sample, only the first case has a 61 percent probability of being an expensive housing area. When you perform predictions using this approach, you also know the probability that your forecast is accurate and can act accordingly, choosing only predictions with the right level of accuracy. (For instance, you might pick only predictions that exceed an 80 percent likelihood.)

Working through Multiclass Regression

In some situations, you must consider more than a simple binary outcome or work with more than just one class. In fact, the world is full of such situations, such as the following:

- Selecting the color of an object from a list of colors
- Identifying an object shape from a list of shapes
- Choosing a particular action from a list of potential actions
- Determining an object type from a list of types

However, when you work through a multiclass regression, what you really do is allow the package to break the multiple classes into multiple binary decisions. The task occurs in the background, where you really don't need to worry about the details, but you could also perform this task manually if necessary. With these understandings in mind, the following sections discuss multiclass regression and various affiliated multiclass and multi-output strategies.

Understanding multiclass regression

All regression eventually breaks down into a binary decision, but your package will hide this fact from view to help you simplify any problem you're trying to solve. In fact, you usually receive support for these kinds of multiclass problems:

- **Multiclass classification:** In this case, you have an object and must decide between multiple classes. For example, when examining a car's color, you must decide whether it's blue, green, brown, or red. The assumption is that the object is assigned to just one class, meaning that the car can't be both blue and green but must appear as blue or green.
- **Multilabel classification:** This problem concerns an object that can have more than one class. For example, a car might have a blue upper half and a contrasting brown lower half. In this case, the car color is both blue and brown. You often see this sort of classification used to tag text. A book might contain humor but also talk about politics and its effects on medicine.

- **Multi-output regression:** In this case, you perform a detailed analysis of complex objects and assign the object more than one target class. The classifications are binary, but the object has multiple properties. For example, you might say that a piece of fruit is an apple or a pear and that its exterior color is either yellow or green. The kind of fruit and its color are two separate target classes that you must handle individually.
- **Multi-output/multiclass classification:** In this case, an object can have multiple values assigned to multiple target classes. For example, you might consider the exterior and interior colors of a car, which you would assign as separate target classes. Another example is weather, when you assign the current weather conditions properties such as temperature range (cold, cool, warm, or hot), humidity (dry, moderate, or humid), wind speed (calm, pleasant, brisk, or hurricane), wind direction (cardinal points of the compass), and so on.
- **Multitask classification:** With enough care, you can perform multiple classifications for each of multiple targets, with each being able to include multiple classes. Obviously, the more complexity you create for a classification and its outputs, the longer the classification takes.



REMEMBER Along with the kind of classification you want to perform, you must also consider the strategy used to perform it. The “[Considering when classes are more](#)” section, earlier in this chapter, discusses strategies in more detail.

Developing a multiclass regression implementation

In the “[Considering when classes are more](#)” section, earlier in this chapter, you see a comparison of strategies. In the following example, you see the OVR strategy used with the Iris dataset, in which a flower can appear as part of the Setosa, Versicolour, or Virginica class (see <https://archive.ics.uci.edu/ml/datasets/iris> for details). The first step is to import the data, as shown here:

```
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
X, y = iris.data, iris.target
```

The `x` data includes columns for sepal length, sepal width, petal length, and petal width, and the `y` data contains the iris class. The next step is to perform the multiclass classification, as shown here:

```
from sklearn.multiclass import OneVsRestClassifier
```

```
from sklearn.svm import LinearSVC
```

```
irisClass = OneVsRestClassifier(LinearSVC(
```

```
    random_state=0)).fit(X, y).predict(X)
```

You can now plot the results based on sepal length and sepal width, using the `irisClass` regression results to provide the colors for the various entries, as shown here:

```
plt.scatter(X[:,0], X[:,1], c=irisClass)
```

```
plt.show()
```



REMEMBER Notice how you use `X[:,0]` and `X[:,1]` to separate out the sepal length (x-axis) and sepal width (y-axis) for the scatterplot. The result is the plot shown in [Figure 2-5](#).

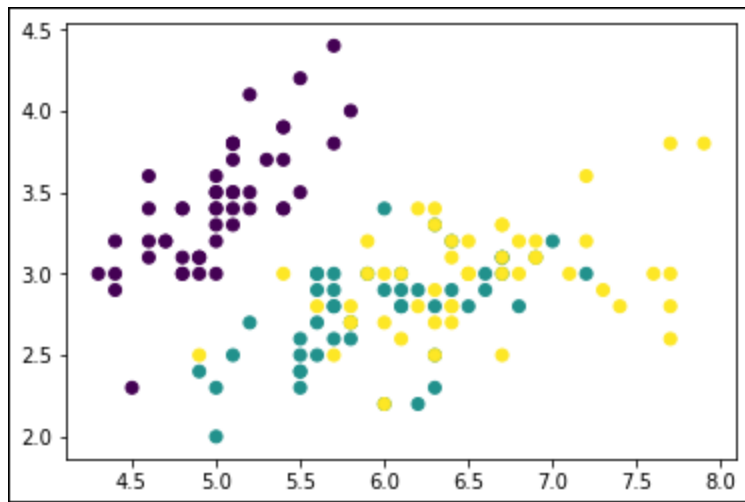


FIGURE 2-5: The plot shows the result of a multiclass regression among three classes.

Table of contents collapsed